A Practical Approach to Data Mining and Visualization

Dr. Mukesh Mann

Head, Department of Information Technology

Faculty Incharge, IT Department

A Practical Approach to Data Mining and Visualization

Author: Dr. Mukesh Mann
Designation: Head, Department of Information Technology

© 2026 Dr. Mukesh Mann. All rights reserved.

This material is prepared for academic use for the course Data Mining and Visualization (B.Tech, 5th Semester).

No part of this material may be reproduced without written permission from the author.

First Edition: August 2026
Printed at: Department of Information Technology

Preface

"Without data, you're just another person with an opinion."
— W. Edwards Deming

Data is the new oil, and the ability to extract value from it has become one of the most sought-after skills of the twenty-first century. We live in an era where every click, every transaction, every heartbeat sensor reading, and every social media post generates data at a scale that was unimaginable two decades ago. The question is no longer *whether* data exists, but *how* we make sense of it.

This book, **A Practical Approach to Data Mining and Visualization**, has been written with a single, unwavering conviction: *theory without practice is incomplete.* Having taught Data Mining and related courses for several years, I have observed that students often struggle not because the concepts are difficult, but because they lack a clear bridge between what is taught in the classroom and what is expected in industry.

Philosophy of This Book

This book follows what I call the **CEMP** teaching philosophy:

- **C** — *Concept first*: Every topic begins with a clear, jargon-free explanation of what it is.
- **E** — *Example-driven*: Abstract ideas are always grounded in real-world, relatable examples.
- **M** — *Mathematics made simple*: Where formulas are involved, they are derived step-by-step, never presented as black boxes.
- **P** — *Practice*: Every chapter ends with working Python code, lab exercises, student activities, and graded questions.

How This Book Is Organized

This volume covers **Unit-I: Introduction to Data Analytics**, organized as follows:

Section 1 — Introduction to Data Analytics covers the nature and sources of data, classification of data into structured, semi-structured, and unstructured types, the five characteristics of Big Data (the 5 V's), the Big Data platform (Hadoop ecosystem), the need for analytics, the evolution of analytic scalability, the distinction between analysis and reporting, analytic processes and tools, and real-world applications.

Section 2 — Data Analytics Lifecycle covers the motivation for a structured lifecycle, the key roles in an analytics project, and all six phases in depth: Discovery, Data Preparation, Model Planning, Model Building, Communicating Results, and Operationalization — each illustrated with Python code and a running case study.

Prerequisites

Students are expected to have basic familiarity with Python programming (loops, functions, lists, dictionaries), elementary statistics (mean, median, variance), and basic SQL.

Software Used

All code in this book is written in **Python 3.10+**. The following libraries are used: `pandas`, `numpy`, `matplotlib`, `seaborn`, and `scikit-learn` — all freely available and open-source. Google Colab (`colab.research.google.com`) can be used to run all examples without any local installation.

Acknowledgements

I would like to express my sincere gratitude to the administration for their continuous support, to my colleagues in the Department of Information Technology for their constructive feedback, and most importantly, to my students, whose curiosity and questions are the true inspiration behind every page of this book.

Dr. Mukesh Mann
Head, Department of Information Technology
August 2026

How to Use This Book

Each chapter in this book is self-contained and follows a consistent pedagogical structure. The following guide explains the special elements used throughout.

Visual Guide to Special Elements

Definition Box (Blue) Provides a precise, formal definition of a new term or concept.

Concept Box (Green) Summarizes the key idea of a topic for quick review.

Example Box (Light Blue) Shows a worked example with step-by-step walk-through.

Note Box (Orange) Highlights important warnings, exceptions, or common mistakes.

Mathematics Box (Red) Presents formulas and calculations with full explanation.

Activity Box (Purple) Contains hands-on classroom or lab activities.

Code Listing (Numbered) Shows Python code with syntax highlighting and line numbers.

Output Box Shows the exact program output students should expect.

Note

Recommended Study Approach: Read the concept, study the example, run the code yourself (do not just read it), then attempt the exercises *before* looking at the answers. Learning data analytics requires active engagement — passive reading alone will not suffice.

Contents

Preface **3**

How to Use This Book **5**

1 Introduction to Data Analytics **13**

1.1 What is Data? . . . 13

1.1.1 The DIKW Hierarchy . . . 14

1.2 Sources and Nature of Data . . . 14

1.2.1 Sources of Data . . . 14

1.2.2 Nature of Data . . . 15

1.3 Classification of Data . . . 15

1.3.1 Structured Data . . . 15

1.3.2 Semi-Structured Data . . . 16

1.3.3 Unstructured Data . . . 16

1.3.4 Comparison of All Three Data Types . . . 17

1.4 Characteristics of Data: The Five V's of Big Data . . . 17

1.4.1 V1 — Volume . . . 18

1.4.2 V2 — Velocity . . . 18

1.4.3 V3 — Variety . . . 18

1.4.4 V4 — Veracity . . . 19

1.4.5 V5 — Value . . . 19

1.5 Introduction to Big Data Platform . . . 19

1.5.1 Traditional Systems versus Big Data Platform . . . 20

1.5.2 The Hadoop Ecosystem . . . 20

1.6 Need for Data Analytics . . . 21

1.6.1 Mathematical Illustration: Student Risk Score . . . 21

1.7 Evolution of Analytic Scalability . . . 22

1.8 Analysis versus Reporting . . . 22

1.8.1 Types of Analytics . . . 23

1.9 Analytic Process and Modern Tools . . . 23
1.9.1 CRISP-DM Process Model . . . 23
1.9.2 Modern Data Analytic Tools . . . 24
1.10 Applications of Data Analytics . . . 24
1.11 Practical Session: Introduction to Python for Data Analytics . . . 24
1.11.1 Setting Up the Environment . . . 24
1.11.2 Exploring Structured Data with Pandas . . . 26
1.12 Data Analytics Lifecycle . . . 28
1.12.1 Need for a Data Analytics Lifecycle . . . 29
1.12.2 Key Roles in Analytics Projects . . . 29
1.12.3 Phase 1: Discovery . . . 29
1.12.4 Phase 2: Data Preparation . . . 30
1.12.5 Phase 3: Model Planning . . . 33
1.12.6 Phase 4: Model Building . . . 34
1.12.7 Phase 5: Communicating Results . . . 39
1.12.8 Phase 6: Operationalization . . . 39
1.13 Student Activities and Exercises . . . 43
1.13.1 Classroom Activities . . . 43
1.13.2 Lab Exercises . . . 46
1.13.3 Mini-Project: Student Academic Risk Predictor . . . 62
1.14 Questions and Answers . . . 63
1.14.1 Short Answer Questions (2 marks each) . . . 63
1.14.2 Long Answer Questions (5–10 marks each) . . . 65
1.14.3 True / False Questions (1 mark each) . . . 67
1.14.4 Fill in the Blanks (1 mark each) . . . 67
1.14.5 Numerical Problems (with Solutions) . . . 68

References and Further Reading **71**

2 Data Preprocessing **73**
2.1 Bridging from the Previous Chapter: Why Raw Data Is Never Ready . 73
2.2 Overview of Data Preprocessing . . . 74
2.2.1 The Four Pillars of Preprocessing . . . 75
2.2.2 Why Preprocessing Matters: The GIGO Principle . . . 75
2.3 Data Cleaning . . . 75
2.3.1 Missing Values . . . 76
2.3.2 Noisy Data . . . 79
2.4 Data Integration . . . 81
2.4.1 Schema Matching . . . 82
2.4.2 The Entity Identification Problem . . . 82

2.5 Redundancy and Correlation Analysis . . . 84
2.5.1 Chi-Square Test for Nominal Attributes . . . 84
2.5.2 Pearson Correlation for Numeric Attributes . . . 85
2.5.3 Covariance . . . 86
2.6 Tuple Duplication . . . 86
2.6.1 Types of Duplicates . . . 87
2.6.2 Why Duplicates Are Harmful . . . 87
2.6.3 Detecting and Removing Duplicates . . . 87
2.7 Data Value Conflict Detection and Resolution . . . 88
2.7.1 Types of Data Value Conflicts . . . 89
2.7.2 Unit and Scale Conflict: Mathematical Resolution . . . 89
2.7.3 Timestamp-Based Conflict Resolution . . . 90
2.7.4 Conflict Resolution Strategies: A Framework . . . 90
2.8 Data Profiling: Knowing Your Data Before Touching It . . . 91
2.8.1 Column-Level Profile Statistics . . . 91
2.8.2 Dataset-Level Profile Metrics . . . 92
2.9 Feature Encoding: Converting Categories into Numbers . . . 93
2.9.1 Label Encoding . . . 93
2.9.2 One-Hot Encoding (OHE) . . . 94
2.9.3 Ordinal Encoding . . . 95
2.10 Feature Scaling and Normalisation . . . 96
2.10.1 Min-Max Normalisation (Rescaling) . . . 96
2.10.2 Z-Score Standardisation . . . 97
2.10.3 Robust Scaling . . . 97
2.10.4 Log and Power Transformations . . . 98
2.10.5 Choosing the Right Scaling Method . . . 99
2.11 Data Discretisation: From Continuous to Categorical . . . 99
2.11.1 Equal-Width Binning . . . 100
2.11.2 Equal-Frequency (Quantile) Binning . . . 100
2.11.3 Entropy-Based (Supervised) Discretisation . . . 100
2.11.4 Comparison of Discretisation Methods . . . 102
2.12 Handling Imbalanced Datasets . . . 102
2.12.1 The Accuracy Paradox . . . 103
2.12.2 Strategy 1: Random Undersampling . . . 103
2.12.3 Strategy 2: Random Oversampling . . . 103
2.12.4 Strategy 3: SMOTE — Synthetic Minority Oversampling Technique 104
2.12.5 Strategy 4: Class Weights . . . 105
2.12.6 Evaluation Metrics for Imbalanced Datasets . . . 106
2.13 Feature Selection: Keeping Only What Matters . . . 106

2.13.1 Why Feature Selection Matters: The Curse of Dimensionality . 107
2.13.2 Method 1: Filter Methods 108
2.13.3 Method 2: Wrapper Methods 108
2.13.4 Method 3: Embedded Methods 109
2.13.5 Comparison of Feature Selection Methods 110
2.14 Data Transformation Pipelines: Putting It All Together 110
2.14.1 The Canonical Preprocessing Order 111
2.14.2 sklearn Pipeline: Automating the Order 111
2.15 Python Practical: Full Preprocessing Pipeline 115
2.15.1 Lab A: Missing Values and Noisy Data 115
2.15.2 Lab B: Integration, Redundancy, Duplicates, and Conflict Resolution 121
2.16 Student Activities and Exercises 128
2.16.1 Classroom Activities 128
2.16.2 Lab Exercises 130
2.16.3 Mini-Project: Real-World Data Cleaning Challenge 136
2.17 Questions and Answers 136
2.17.1 Short Answer Questions (2 marks each) 136
2.17.2 Long Answer Questions (5–10 marks each) 139
2.17.3 True / False Questions (1 mark each) 141
2.17.4 Fill in the Blanks (1 mark each) 141
2.17.5 Numerical Problems (with Solutions) 142
Unit II: Additional References 144

3 Data Reduction and Data Transformation 145
3.1 Bridging from the Previous Chapters: The Reduction Imperative 146
3.2 Overview of Data Reduction Strategies 146
3.2.1 Taxonomy of Reduction Strategies 147
3.2.2 The Reduction-Quality Trade-off 147
3.2.3 Master Decision Guide: Which Reduction Strategy to Choose? . 148
3.2.4 Before vs. After Reduction: Performance Benchmark 148
3.2.5 Common Pitfalls and Anti-Patterns in Data Reduction 149
3.3 Wavelet Transforms 149
3.3.1 The Haar Wavelet: Simplest and Most Intuitive 150
3.3.2 Why Wavelets Are Powerful for Data Mining 151
3.4 Principal Component Analysis (PCA) 152
3.4.1 The Geometric Intuition of PCA 152
3.4.2 The Mathematical Derivation of PCA 153
3.4.3 The Scree Plot: Choosing k 155

3.4.4 When to Use PCA and When Not To . . . 155
3.4.5 PCA in Practice: Step-by-Step Decision Guide . . . 155
3.5 Attribute Subset Selection . . . 156
3.5.1 The Feature Selection Search Problem . . . 156
3.5.2 Evaluation Criteria for Subset Quality . . . 157
3.6 Parametric Data Reduction: Regression Models . . . 157
3.6.1 Linear Regression for Data Reduction . . . 158
3.6.2 Multiple Linear Regression . . . 159
3.6.3 Log-Linear Models . . . 159
3.7 Numerosity Reduction: Histograms . . . 160
3.7.1 Types of Histograms . . . 160
3.7.2 How to Use Histograms for Approximate Query Answering . . . 161
3.8 Numerosity Reduction: Clustering . . . 161
3.8.1 k-Means Clustering for Data Reduction . . . 162
3.8.2 BIRCH: Clustering for Very Large Datasets . . . 162
3.9 Numerosity Reduction: Sampling . . . 163
3.9.1 Simple Random Sampling Without Replacement (SRSWOR) . . . 163
3.9.2 Stratified Sampling . . . 164
3.9.3 Cluster Sampling and Systematic Sampling . . . 165
3.10 Data Cube Aggregation . . . 165
3.10.1 Cube Operations: Roll-Up, Drill-Down, Slice, Dice, Pivot . . . 166
3.10.2 The Data Cube: OLAP vs. OLTP . . . 167
3.11 Data Transformation by Normalisation . . . 167
3.11.1 The Three Standard Normalisation Methods . . . 168
3.11.2 Comprehensive Normalisation Decision Framework . . . 169
3.11.3 The Effect of Normalisation on Algorithms . . . 169
3.12 Discretisation in the Reduction Context . . . 170
3.12.1 Discretisation by Histogram Analysis . . . 170
3.12.2 Discretisation by Clustering . . . 171
3.12.3 Discretisation by Decision Tree Analysis . . . 171
3.12.4 Discretisation by Correlation Analysis . . . 172
3.13 Concept Hierarchy Generation for Nominal Data . . . 173
3.13.1 Types of Concept Hierarchies . . . 173
3.13.2 Automatic Hierarchy Generation from Data . . . 173
3.13.3 Multi-Level Mining with Concept Hierarchies . . . 174
3.14 Python Lab: Data Reduction and Transformation . . . 175
3.14.1 Lab A: PCA and Regression Data Reduction . . . 175
3.14.2 Lab B: Sampling, Normalisation, and Discretisation . . . 180
3.15 Student Activities . . . 187

3.16 Questions and Answers 188
3.16.1 Short Answer Questions (2 marks each) 189
3.16.2 Long Answer Questions (5–10 marks each) 191
3.16.3 Numerical Problems (with Full Solutions) 192
3.16.4 True / False Questions 194
Chapter 3: Additional References 195

CHAPTER 1

Introduction to Data Analytics

"Data is a precious thing and will last longer than the systems themselves." — Tim Berners-Lee

Chapter Overview

- What is Data? Nature and Sources of Data
- Classification: Structured, Semi-Structured, Unstructured
- Characteristics of Data: The Five V's of Big Data
- Introduction to Big Data Platform (Hadoop Ecosystem)
- Need for Data Analytics
- Evolution of Analytic Scalability
- Analysis versus Reporting
- Types of Analytics (Descriptive to Prescriptive)
- Analytic Process and Tools (CRISP-DM)
- Applications of Data Analytics

1.1 What is Data?

Key Concept: Opening Thought

Ask yourself: How many apps did you open this morning before reaching college — WhatsApp, YouTube, Google Maps, UPI? Every single tap you made generated **DATA**. Welcome to the Data Age.

Definition: Data

Data is a collection of raw, unprocessed facts, figures, symbols, or observations. Data has no inherent meaning on its own until it is processed and placed in context.

1.1.1 The DIKW Hierarchy

The progression from raw data to actionable wisdom is captured in the **DIKW** (Data–Information–Knowledge–Wisdom) hierarchy.

Table 1.1: The DIKW Hierarchy with College Examples

Level	Definition	College Example
Data	Raw, unprocessed facts	`28, Male, Delhi, 8.4, CSE`
Information	Processed data with meaning	"28-year-old CSE male from Delhi with CGPA 8.4"
Knowledge	Information + context + experience	"Urban high-CGPA males prefer industry internships"
Wisdom	Applied knowledge for decisions	College establishes industry tie-ups for placements

Example: Classroom Analogy

Data is like **raw wheat**. Information is **flour**. Knowledge is **bread**. Wisdom is running a **bakery business**. Each step adds processing and value.

1.2 Sources and Nature of Data

1.2.1 Sources of Data

Data is generated from a wide variety of sources in the modern world. Understanding these sources helps analysts know where to look for data and what quality to expect.

Table 1.2: Data Sources with College Examples

Source Type	College Example	Industry Example
Internal	Student ERP (attendance, marks)	Company sales database
Internal	Faculty evaluation records	HR performance records
External	AICTE / UGC open data portal	Government census data
External	Twitter/X posts about the college	Competitor pricing data
Machine	Biometric gate entry scanner	IoT factory sensors
Machine	Library RFID borrow/return logs	GPS vehicle traces

1.2.2 Nature of Data

Table 1.3: Nature of Data

Nature	Description	Example
Quantitative	Numeric, measurable values	Marks: 78, 92, 65; CGPA: 8.4
Qualitative	Descriptive, categorical	Branch: CSE, IT, ECE
Temporal	Time-based observations	Attendance on 15-Aug-2026
Spatial	Location-based data	Student home city GPS coords

1.3 Classification of Data

Key Concept: The Classification Analogy

Think of students in a college:

- **Structured** = Students with proper ID cards (organized in rows and columns)
- **Semi-structured** = Students with some info on paper, some on phone (partial)
- **Unstructured** = A crowd at the college fest with no list (no fixed format)

1.3.1 Structured Data

Definition: Structured Data

Structured data is organized in a predefined format of rows and columns conforming to a fixed schema. It is easily searchable, stored in relational databases (RDBMS), and represents approximately **20%** of all data generated in the world today.

Example: Structured Data — Student Academic Records

Roll No	Name	Branch	CGPA	Marks	City
2021001	Ravi Kumar	CSE	8.4	78	Rohtak
2021002	Priya Sharma	IT	9.1	91	Sonepat
2021003	Amit Singh	ECE	7.8	72	Panipat
2021004	Sunita Devi	CSE	6.5	60	Delhi

This data is stored directly in an SQL table and queried with: `SELECT * FROM students WHERE CGPA > 8.0`

1.3.2 Semi-Structured Data

Definition: Semi-Structured Data

Semi-structured data does not conform to the rigid tabular model but contains tags, markers, or key-value pairs that provide organizational structure. Common formats include **JSON**, **XML**, and **YAML**. Most modern web APIs return semi-structured data.

```
# This is how a student's profile looks in JSON format.
# Notice the nested structure -- 'subjects' is a list,
# and 'scores' is a nested dictionary inside the main object.
# This CANNOT be stored in a single SQL row directly.

{
  "student": {
    "roll_no"  : "2021002",
    "name"     : "Priya Sharma",
    "branch"   : "IT",
    "cgpa"     : 9.1,
    "subjects" : ["Data Mining", "Operating Systems", "CN"],
    "scores"   : {
      "Data_Mining"      : 91,
      "Operating_Systems": 88,
      "CN"               : 85
    },
    "activities": ["NSS", "Dance Club"]
  }
}
```

Listing 1.1: JSON representation of a student profile

1.3.3 Unstructured Data

Definition: Unstructured Data

Unstructured data has no predefined format or schema. It is the most abundant form of data in the world, accounting for approximately **80%** of all data generated. It cannot be directly stored in traditional relational databases.

Example: Unstructured Data — Student Feedback

> *"The Data Mining lab sessions are really helpful. Dr. Mann explains the concepts very clearly with real examples. However, I think the lab computers need to be upgraded — Python runs very slowly on the old systems. Overall, I would rate this course 9 out of 10!"*

This feedback **cannot** be stored in a structured table. To analyze it, we need **Natural Language Processing (NLP)** techniques.

1.3.4 Comparison of All Three Data Types

Table 1.4: Structured vs. Semi-Structured vs. Unstructured Data

Feature	Structured	Semi-Structured	Unstructured
Format	Fixed (rows/cols)	Flexible with tags	No fixed format
Storage	RDBMS	NoSQL/XML/JSON	File systems/Hadoop
% of World Data	∼20%	∼10–15%	∼65–80%
Query Language	SQL	XQuery/JSONPath	NLP/ML algorithms
Example	Excel marks sheet	WhatsApp backup JSON	Student feedback PDF
Tools	MySQL, Oracle	MongoDB, CouchDB	Hadoop, Spark, NLP

Note

Industry Reality: Most real-world data analytics projects deal with all three types simultaneously. A typical e-commerce platform handles structured (orders, prices), semi-structured (product catalogs in JSON), and unstructured (customer reviews) data all at once.

1.4 Characteristics of Data: The Five V's of Big Data

Key Concept: Why Five V's?

Not all data is equal. When data becomes "big," traditional tools fail. The **Five V's** framework characterizes what makes data Big and which aspects present unique challenges.

1.4.1 V1 — Volume

Definition: Volume

Volume refers to the sheer *amount* of data generated and stored. Big Data typically refers to datasets measured in terabytes (TB), petabytes (PB), or even exabytes (EB).

Scale reference:

$$1 \text{ KB} \rightarrow 1 \text{ MB} \rightarrow 1 \text{ GB} \rightarrow 1 \text{ TB} \rightarrow 1 \text{ PB} \rightarrow 1 \text{ EB}$$

Each step is 10^3 ($\approx 1000\times$) larger than the previous.

Examples: Facebook generates >500 TB/day. Aadhaar database holds biometric data for 1.4 billion citizens.

1.4.2 V2 — Velocity

Definition: Velocity

Velocity refers to the *speed* at which data is generated, collected, and must be processed. Modern systems often require *real-time* or *near-real-time* processing.

Examples: Twitter generates ∼500,000 tweets per minute. IRCTC processes over 1 lakh booking requests per minute during Tatkal opening. A university biometric gate generates 2,000+ entry/exit events per day during peak hours.

1.4.3 V3 — Variety

Definition: Variety

Variety refers to the *diversity of data types and formats*. Big Data comes from multiple sources in multiple formats simultaneously — structured (numbers), semi-structured (JSON/XML), and unstructured (text, video, audio).

1.4.4 V4 — Veracity

Definition: Veracity

Veracity refers to the *accuracy, trustworthiness, and quality* of data. Big Data is often noisy, inconsistent, or incomplete. Low veracity data leads to wrong conclusions — "garbage in, garbage out."

College example of low veracity: Proxy attendance marked by a friend; a CGPA of 15.5 entered in a scale of 10; duplicate student records.

Industry example: Aadhaar introduced biometric authentication to solve the veracity problem of ghost beneficiaries in government schemes.

1.4.5 V5 — Value

Definition: Value

Value is the most important V. It refers to the *useful insights and business benefit* that can be extracted from data. Data without value is just storage cost.

Example: Netflix offered $1 million in 2006 to anyone who could improve their recommendation algorithm by 10%. Better recommendations = more watch time = more subscribers = more revenue. That is *value from data.*

Table 1.5: The Five V's of Big Data

V	Description	College Example	Industry Example
Volume	Amount of data	5 years attendance records, 2000 students	Facebook: 500 TB/day
Velocity	Speed of data	Real-time biometric gate	IRCTC: 1L requests/min
Variety	Types/formats	Marks + feedback + photos	Medical: scans + prescriptions
Veracity	Data quality	Proxy attendance	Amazon: fake reviews
Value	Actionable insight	Early warning for at-risk students	Netflix recommendations

1.5 Introduction to Big Data Platform

1.5.1 Traditional Systems versus Big Data Platform

Table 1.6: Traditional RDBMS vs. Big Data Platform

Feature	Traditional (RDBMS)	Big Data Platform
Data Size	Gigabytes	Terabytes to Petabytes
Processing	Single machine	Cluster of commodity machines
Data Type	Structured only	All types
Processing Mode	Batch	Real-time + Batch
Fault Tolerance	Low	High (built-in replication)
Cost	High (licensed)	Low (open-source Hadoop)
Example	College ERP system	YouTube video analytics

1.5.2 The Hadoop Ecosystem

Apache Hadoop is the dominant open-source framework for Big Data. Its four core components are:

1. **HDFS** (Hadoop Distributed File System) — Splits files into 128 MB blocks, distributes and replicates them across machines.
2. **MapReduce** — A two-stage processing model: *Map* (distribute and process) + *Reduce* (aggregate).
3. **YARN** (Yet Another Resource Negotiator) — Manages cluster resources and schedules jobs.
4. **Hadoop Common** — Shared utilities and libraries.

Mathematics: How MapReduce Counts Words Across 1 Billion Documents

Problem: Count how many times each word appears.

Step 1 – Map: Each machine processes its share of documents and emits key-value pairs:

$$(\text{"data"}, 1),\ (\text{"mining"}, 1),\ (\text{"data"}, 1),\ \ldots$$

Step 2 – Shuffle: All pairs with the same key are grouped:

$$\text{"data"} \rightarrow [1, 1, 1, \ldots, 1]$$

Step 3 – Reduce: Each group is aggregated:

$$\text{"data"} \rightarrow \sum_{i=1}^{n} 1 = 15{,}432$$

This entire computation runs across 1,000 machines in *parallel*, completing in minutes instead of days.

1.6 Need for Data Analytics

Table 1.7: Problems Solved by Data Analytics

Without Data Analytics	With Data Analytics
"Why are sales dropping?" → Guessing	Pinpoint the exact product, region, and time
"Which student will fail?" → Unknown	Analyze attendance + marks → Early warning system
"Is our ad working?" → Hope	Click rate + conversion analysis → Exact ROI
"What to stock?" → Intuition	Demand forecasting → Optimal inventory

1.6.1 Mathematical Illustration: Student Risk Score

Mathematics: Computing a Simple Student Risk Score

Formula (Weighted Linear Combination):

$$R = w_1 \cdot \left(1 - \frac{A}{100}\right) + w_2 \cdot \left(1 - \frac{M}{100}\right)$$

where A = attendance %, M = internal marks %, $w_1 = 0.6$, $w_2 = 0.4$, and $w_1 + w_2 = 1$.

Classification thresholds: $R < 0.20$ = Low Risk, $0.20 \leq R < 0.35$ = Medium Risk, $R \geq 0.35$ = High Risk.

Student A: $A = 60\%$, $M = 55\%$

$$R_A = 0.6(1 - 0.60) + 0.4(1 - 0.55) = 0.24 + 0.18 = \mathbf{0.42} \quad \Rightarrow \text{HIGH RISK}$$

Student B: $A = 80\%$, $M = 55\%$

$$R_B = 0.6(0.20) + 0.4(0.45) = 0.12 + 0.18 = \mathbf{0.30} \quad \Rightarrow \text{MEDIUM RISK}$$

Student C: $A = 88\%$, $M = 82\%$

$$R_C = 0.6(0.12) + 0.4(0.18) = 0.072 + 0.072 = \mathbf{0.144} \quad \Rightarrow \text{LOW RISK}$$

1.7 Evolution of Analytic Scalability

Table 1.8: Evolution of Analytic Scalability

Era	Technology	Data Scale	Example
1960s	Flat Files, COBOL	KBs	Bank ledgers
1980s	RDBMS, SQL	MBs–GBs	Hospital records
1990s	Data Warehouses, OLAP	GBs	Retail reports
2000s	Hadoop, NoSQL	TBs	Web search logs
2010s	Spark, Cloud	PBs	Social media analytics
Now	AI + Streaming	Exabytes	Self-driving cars

Vertical Scaling (Scale Up): Add more RAM/CPU to one machine. *Expensive and physically limited.*

Horizontal Scaling (Scale Out): Add more machines to a cluster. *Cheap, fault-tolerant, and theoretically unlimited.* This is the approach used by Hadoop and Spark.

1.8 Analysis versus Reporting

Key Concept: The Critical Distinction

Reporting tells you *what* happened. **Analysis** tells you *why* it happened and what *will* happen. This distinction defines the boundary between a report writer and a data analyst.

Table 1.9: Analysis versus Reporting

Feature	Reporting	Analysis
Purpose	Summarize past data	Discover patterns, causes, future trends
Question	What happened?	Why? What will happen? What to do?
Output	Tables, charts, dashboards	Insights, predictions, recommendations
Who uses	Managers, executives	Data scientists, analysts
Tools	Excel, Tableau, Power BI	Python, R, Machine Learning
Frequency	Scheduled (daily/weekly)	On-demand, exploratory

Example: Reporting vs. Analysis — Student Results

Reporting: "In Semester 5, 120 students scored below 6 CGPA in Data Mining."
Analysis: "Students who attended $< 70\%$ of DM labs AND scored $< 50\%$ in Unit-1 are $3\times$ more likely to fail the end-semester exam. *Recommendation:* Mandatory remedial lab sessions before Unit-2."

1.8.1 Types of Analytics

Table 1.10: The Four Types of Analytics

Type	Question	Example	Complexity
Descriptive	What happened?	Monthly attendance report	Low (easy)
Diagnostic	Why did it happen?	Why did 40 students fail?	Medium
Predictive	What will happen?	Who will fail next year?	High
Prescriptive	What should we do?	Which students to tutor?	Highest

1.9 Analytic Process and Modern Tools

1.9.1 CRISP-DM Process Model

The **CRISP-DM** (Cross Industry Standard Process for Data Mining) is the most widely adopted framework for structuring data mining projects. Its six iterative phases are:

1. **Business Understanding**: Understand the problem domain and define analytical objectives
2. **Data Understanding**: Collect initial data, explore, and identify data quality issues
3. **Data Preparation**: Clean, transform, and prepare the data for modeling
4. **Modeling**: Select and apply modeling techniques; calibrate parameters
5. **Evaluation**: Evaluate the model against business objectives; decide on deployment
6. **Deployment**: Deliver results to end users; automate the model in production

Note

CRISP-DM is **iterative**, not linear. Results from Evaluation can send the project back to Data Understanding or even Business Understanding. This is normal and healthy in data science projects.

1.9.2 Modern Data Analytic Tools

Table 1.11: Modern Data Analytic Tools by Category

Category	Tools	Best For
Programming	Python, R, Scala	Custom analysis, ML models
BI/Reporting	Power BI, Tableau, Looker	Dashboards, visual reports
Big Data	Hadoop, Apache Spark	Large-scale processing
Databases	SQL, MongoDB, Cassandra	Data storage, retrieval
Cloud	AWS, Google Cloud, Azure	Scalable infrastructure
ML Libraries	scikit-learn, TensorFlow	Machine learning
Notebooks	Jupyter, Google Colab	Interactive exploration

1.10 Applications of Data Analytics

Table 1.12: Applications of Data Analytics Across Domains

Domain	Application	Impact
Healthcare	Disease outbreak prediction; readmission risk	Save lives
Finance	Fraud detection, credit scoring	Prevent losses
Retail	Demand forecasting, recommendation engine	Increase revenue
Education	Student performance prediction, dropout alert	Improve outcomes
Sports	Player performance analysis (Moneyball)	Winning strategy
Government	Crime prediction, traffic management	Public safety
Agriculture	Crop yield prediction, irrigation optimization	Food security
Social Media	Sentiment analysis, ad targeting	Engagement

1.11 Practical Session: Introduction to Python for Data Analytics

1.11.1 Setting Up the Environment

Open a terminal (Command Prompt or Anaconda Prompt) and run:

Listing 1.2: Installing required Python libraries

```
pip install pandas numpy matplotlib seaborn scikit-learn
   openpyxl jupyter
```

Then verify with:

```
#
    ============================================================
```

```python
# lab1_verify.py
# Purpose : Check if all required libraries are installed
# Author  : Dr. Mukesh Mann
# Course  : Data Mining and Visualization
#
    ============================================================

import sys

# List of libraries required for this course
libraries = ['pandas', 'numpy', 'matplotlib', 'seaborn',
    'sklearn']

print("=" * 55)
print("  DATA MINING -- ENVIRONMENT CHECK")
print("=" * 55)
print(f"  Python Version : {sys.version[:6]}")
print("-" * 55)

for lib in libraries:
    try:
        # Try to import the library
        mod     = __import__(lib)
        version = getattr(mod, '__version__', 'OK')
        print(f"  OK   {lib:<18} installed  (v{version})")
    except ImportError:
        # Library not found
        print(f"  ERR  {lib:<18} NOT FOUND -- run: pip
    install {lib}")

print("=" * 55)
print("  Setup complete! Ready for Data Mining.")
print("=" * 55)
```

Listing 1.3: Environment verification script (lab1_verify.py)

Program Output

```
=======================================================
  DATA MINING -- ENVIRONMENT CHECK
=======================================================
  Python Version : 3.10.x
-------------------------------------------------------
```

```
  OK   pandas             installed  (v2.1.0)
  OK   numpy              installed  (v1.25.2)
  OK   matplotlib         installed  (v3.7.2)
  OK   seaborn            installed  (v0.12.2)
  OK   sklearn            installed  (v1.3.0)
============================================================
  Setup complete! Ready for Data Mining.
============================================================
```

1.11.2 Exploring Structured Data with Pandas

```
#
    ================================================================
# lab1_structured.py
# Purpose : Load, explore, filter, and group student data
# Author  : Dr. Mukesh Mann
#
    ================================================================

import pandas as pd
import numpy as np

# ---- STEP 1: Create the dataset ----
# Each key is a column; each list is that column's values
data = {
    'Roll_No'       : [1001, 1002, 1003, 1004, 1005,
                       1006, 1007, 1008, 1009, 1010],
    'Name'          :
   ['Ravi','Priya','Amit','Sunita','Rajesh',

   'Kavya','Rohit','Neha','Vikram','Anita'],
    'Branch'        : ['CSE','IT','CSE','ECE','IT',
                       'CSE','ECE','IT','CSE','IT'],
    'Attendance_%' : [85, 62, 90, 78, 45, 92, 70, 55, 88,
   95],
    'Marks'         : [72, 48, 85, 65, 30, 90, 58, 42, 78,
   92],
    'CGPA'          : [7.8, 6.2, 8.5, 7.0, 5.5,
                       9.0, 6.8, 5.8, 8.0, 9.2]
}

# ---- STEP 2: Convert dictionary to a Pandas DataFrame ----
```

```
df = pd.DataFrame(data)

# ---- STEP 3: Explore the dataset ----
print("=== DATASET ===")
print(df.to_string(index=False))

print("\n=== SHAPE (rows, columns) ===")
print(df.shape)              # Output: (10, 6)

print("\n=== STATISTICAL SUMMARY ===")
# describe() shows count, mean, std, min, 25%, 50%, 75%, max
print(df.describe().round(2))

print("\n=== MISSING VALUES ===")
print(df.isnull().sum())    # Should show 0 for all columns

# ---- STEP 4: Filtering -- find students at risk ----
print("\n=== STUDENTS WITH ATTENDANCE < 75% ===")
low_att = df[df['Attendance_%'] < 75]
print(low_att[['Name', 'Branch', 'Attendance_%', 'Marks']])

# ---- STEP 5: Grouping -- average CGPA per branch ----
print("\n=== AVERAGE CGPA PER BRANCH ===")
print(df.groupby('Branch')['CGPA'].mean().round(2))
```

Listing 1.4: Exploring structured student data (lab1_structured.py)

Program Output

```
=== DATASET ===
 Roll_No    Name Branch  Attendance_%  Marks  CGPA
    1001    Ravi    CSE          85     72   7.8
    1002   Priya     IT          62     48   6.2
    1003    Amit    CSE          90     85   8.5
    1004  Sunita    ECE          78     65   7.0
    1005  Rajesh     IT          45     30   5.5
    1006   Kavya    CSE          92     90   9.0
    1007   Rohit    ECE          70     58   6.8
    1008    Neha     IT          55     42   5.8
    1009  Vikram    CSE          88     78   8.0
    1010   Anita     IT          95     92   9.2

=== SHAPE (rows, columns) ===
(10, 6)
```

```
=== STATISTICAL SUMMARY ===
       Roll_No  Attendance_%   Marks   CGPA
count     10.0          10.0    10.0   10.0
mean    1005.5          76.0    66.0    7.38
std        3.0          17.3    21.3    1.22
min     1001.0          45.0    30.0    5.50
50%     1005.5          79.0    68.5    7.40
max     1010.0          95.0    92.0    9.20

=== STUDENTS WITH ATTENDANCE < 75% ===
    Name Branch  Attendance_%  Marks
   Priya     IT            62     48
  Rajesh     IT            45     30
   Rohit    ECE            70     58
    Neha     IT            55     42

=== AVERAGE CGPA PER BRANCH ===
Branch
CSE    8.325
ECE    6.900
IT     6.940
```

1.12 Data Analytics Lifecycle

"87% of data science projects never make it to production. The main reason? No structured process." — VentureBeat, 2019

Chapter Overview

- Need for a Data Analytics Lifecycle
- Key Roles in Successful Analytics Projects
- Phase 1: Discovery
- Phase 2: Data Preparation (ETL/ELT)
- Phase 3: Model Planning
- Phase 4: Model Building and Evaluation
- Phase 5: Communicating Results
- Phase 6: Operationalization and Deployment
- Complete Python Practical: All 6 Phases

1.12.1 Need for a Data Analytics Lifecycle

Key Concept: Why a Lifecycle?

Every engineering discipline has a structured process: software has SDLC, construction follows PMI standards, clinical trials follow FDA protocols. Without a structured lifecycle, data analytics projects become chaotic, results are unreproducible, and stakeholders lose confidence.

The Data Analytics Lifecycle consists of **six phases**:

1. Discovery
2. Data Preparation
3. Model Planning
4. Model Building
5. Communicating Results
6. Operationalization

1.12.2 Key Roles in Analytics Projects

Table 1.13: Key Roles in a Data Analytics Project

Role	Responsibilities	Key Skills
Business Analyst	Translate business needs into data questions	Domain, communication
Data Engineer	Build data pipelines and infrastructure	SQL, Python, Spark
Data Scientist	Build and evaluate predictive models	Python/R, ML, Statistics
Data Analyst	Analyze data and generate reports	SQL, Excel, Tableau
Data Visualizer	Create dashboards and visual stories	Design, Power BI
Project Manager	Oversee timeline, resources, stakeholders	Leadership, planning

1.12.3 Phase 1: Discovery

Definition: Discovery

The **Discovery** phase involves understanding the business problem, the goals, the available data, and framing a testable hypothesis — *before any data is touched.* It is the most critical and most neglected phase.

Key activities in Discovery:

1. Define the problem statement clearly
2. Identify stakeholders and their decision-making needs
3. Inventory available data sources
4. Frame the analytical hypothesis
5. Define success criteria (measurable)
6. Assess resource constraints (time, budget, team)

Example: Discovery Phase: Student Risk Predictor

Item	Detail
Problem	High rate of students failing 5th semester exams
Goal	Identify at-risk students by end of Week 8
Hypothesis	Students with attendance $< 75\%$ AND marks $< 50\%$ are at significantly higher risk
Data Sources	ERP (attendance), faculty marks, previous CGPA
Success	Correctly identify $\geq 80\%$ of at-risk students
Timeline	Results needed before internal test-2 (Week 10)

1.12.4 Phase 2: Data Preparation

Definition: Data Preparation

Data Preparation (ETL — Extract, Transform, Load) is the process of collecting, cleaning, transforming, and integrating data from multiple sources into an analysis-ready dataset. It typically consumes **60–80%** of the total project time.

Common Data Quality Problems

Table 1.14: Data Quality Issues and Their Fixes

Problem	Example	Fix
Missing values	CGPA field is NULL (empty)	Fill with mean or median
Duplicates	Same student registered twice	Drop duplicate rows
Wrong format	Date as “18-08-26” vs “2026-08-18”	Standardize format
Outliers	CGPA = 15.5 (impossible on 10 scale)	Remove or correct
Inconsistency	“M”, “Male”, “male” in Gender	Standardize to one value

Mathematics: Handling Missing Values

Mathematics: Mean Imputation vs. Median Imputation

Given CGPA values: [7.5, 8.0, NULL, 6.5, 9.0]

Method 1 — Mean Imputation:

$$\bar{x} = \frac{7.5 + 8.0 + 6.5 + 9.0}{4} = 7.75 \quad \Rightarrow \text{Replace NULL with } 7.75$$

Method 2 — Median Imputation:

Sort non-null values: [6.5, 7.5, 8.0, 9.0]

$$\text{Median} = \frac{7.5 + 8.0}{2} = 7.75 \quad \Rightarrow \text{Replace NULL with } 7.75$$

Rule: Use **Mean** for symmetric data; use **Median** for skewed data or when outliers are present (the median is not affected by extreme values).

```
#
    ================================================================
# phase2_cleaning.py
# Purpose : Demonstrate common data cleaning operations
# Author  : Dr. Mukesh Mann
#
    ================================================================

import pandas as pd
import numpy as np

# ---- Create a purposely messy dataset ----
# This simulates real-world dirty data you will encounter
dirty_data = pd.DataFrame({
    'Name'      : ['Ravi', 'PRIYA', 'Amit', 'ravi', None,
   'Sunita'],
    'Marks'     : [78,      52,      -10,   78,     92,
   150],
    'Attendance': [85,      62,      90,    85,     None,
   74],
    'CGPA'      : [7.8,     6.1,     9.0,  7.8,    8.2,
   15.5]
})

print("=== RAW DIRTY DATASET ===")
```

```
print(dirty_data.to_string())

# ---- Data Quality Audit ----
print("\n=== AUDIT: MISSING VALUES ===")
print(dirty_data.isnull().sum())

print(f"\nDuplicate rows    :
   {dirty_data.duplicated().sum()}")
print(f"Marks out of range: "
      f"{((dirty_data['Marks'] < 0) | (dirty_data['Marks'] >
   100)).sum()}")
print(f"CGPA > 10          : {(dirty_data['CGPA'] >
   10).sum()}")

# ---- Fix the data ----
clean = dirty_data.copy()

# Fix 1: Normalize names to Title Case and fill missing
clean['Name'] = clean['Name'].str.title().fillna('Unknown')

# Fix 2: Clip Marks to valid range [0, 100]
clean['Marks'] = clean['Marks'].clip(0, 100)

# Fix 3: Clip CGPA to valid range [0, 10]
clean['CGPA']  = clean['CGPA'].clip(0, 10)

# Fix 4: Fill missing Attendance with the column median
# (median is preferred because outliers may exist)
median_att = clean['Attendance'].median()
clean['Attendance'] = clean['Attendance'].fillna(median_att)

# Fix 5: Remove duplicate rows
clean = clean.drop_duplicates()

print("\n=== CLEANED DATASET ===")
print(clean.to_string())
print(f"\nMissing values remaining:
   {clean.isnull().sum().sum()}")
```

Listing 1.5: Data cleaning with Python (phase2_cleaning.py)

Program Output

```
=== RAW DIRTY DATASET ===
     Name  Marks  Attendance  CGPA
0    Ravi     78        85.0   7.8
1   PRIYA     52        62.0   6.1
2    Amit    -10        90.0   9.0   <- Marks = -10  (INVALID)
3    ravi     78        85.0   7.8   <- Duplicate of row 0
4    None     92         NaN   8.2   <- Name missing, Attendance missing
5  Sunita    150        74.0  15.5   <- Marks=150, CGPA=15.5 (INVALID)

=== AUDIT: MISSING VALUES ===
Name          1
Marks         0
Attendance    1
CGPA          0

Duplicate rows    : 1
Marks out of range: 2
CGPA > 10         : 1

=== CLEANED DATASET ===
     Name  Marks  Attendance  CGPA
0    Ravi     78        85.0   7.8
1   Priya     52        62.0   6.1
2    Amit      0        90.0   9.0
4  Unknown    92        74.0   8.2
5  Sunita    100        74.0  10.0

Missing values remaining: 0
```

1.12.5 Phase 3: Model Planning

Definition: Model Planning

In **Model Planning**, the data scientist selects the appropriate algorithm, identifies the input features and target variable, and defines the validation strategy — all *before* writing model code.

Algorithm Selection Logic:

- If the output is known (labelled data) → **Supervised Learning**
  - Output is a number (continuous) → **Regression**
  - Output is a category (class) → **Classification**
- If the output is unknown (no labels) → **Unsupervised Learning**

- Known number of groups → **K-Means Clustering**
- Unknown number of groups → **Hierarchical Clustering**

For our student risk predictor: The output (Pass=1 / Fail=0) is known and categorical → **Classification** → **Decision Tree Classifier** selected.

1.12.6 Phase 4: Model Building

The Confusion Matrix and Performance Metrics

Mathematics: Confusion Matrix: All Performance Metrics with Significance

Step 1 — Confusion Matrix (binary classification: Pass/Fail):

	Predicted: PASS	**Predicted: FAIL**
Actual: PASS	**TP = 45** (True Positive)	**FN = 8** (False Negative)
Actual: FAIL	**FP = 5** (False Positive)	**TN = 42** (True Negative)

Terminology:

- **TP (True Positive):** Model predicted PASS and student actually passed. *Correct, desired outcome.*
- **TN (True Negative):** Model predicted FAIL and student actually failed. *Correct, desired outcome.*
- **FP (False Positive):** Model predicted PASS but student actually failed. Also called **Type I Error**.
- **FN (False Negative):** Model predicted FAIL but student actually passed. Also called **Type II Error**.

Step 2 — Performance Metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{45 + 42}{100} = \mathbf{87}\%$$

Significance of Accuracy: Overall correctness of the model. However, it can be **misleading on imbalanced datasets**. If 95 out of 100 students pass, a model that always predicts PASS gets 95% accuracy but catches zero at-risk students.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{45}{50} = \mathbf{90.0}\%$$

Significance of Precision: "Of all students we flagged as passing, how many actually passed?" High precision means **fewer false alarms**. Use when the cost of a False Positive is high (e.g., wasting resources on students who do not need intervention).

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} = \frac{45}{53} \approx \mathbf{84.9}\%$$

Significance of Recall: "Of all students who actually passed, how many did we correctly identify?" High recall means **fewer missed cases**. Use when the cost of a False Negative is high — in education, **missing an at-risk student (FN)** who then fails without intervention is the most dangerous error. In medical diagnosis, a missed cancer patient (FN) is far worse than an unnecessary biopsy (FP). Therefore, in safety-critical applications, **Recall is often more important than Precision**.

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{42}{47} \approx \mathbf{89.4}\%$$

Significance of Specificity: "Of all students who actually failed, how many did we correctly identify?" Complements Recall. High specificity means the model does not incorrectly label failing students as passing.

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 0.90 \times 0.849}{0.90 + 0.849} \approx \mathbf{87.4}\%$$

Significance of F1-Score: The **harmonic mean** of Precision and Recall. It is the best single metric when you need to **balance both**, especially on imbalanced datasets. If either Precision or Recall is very low, F1 will penalize the model severely.

Metric	When to Prioritize	Context Example
Accuracy	Balanced, equal-class datasets	Balanced exam dataset
Precision	FP is costly	Spam filter (not block real email)
Recall	FN is costly (miss = danger)	Disease detection, dropout alert
F1-Score	FP and FN both matter	Fraud detection, NLP tasks
Specificity	Ruling out negatives correctly	COVID screening (no false negatives)

Key Takeaway on Recall: In the student risk prediction scenario, an FN (predicting a student will pass when they will actually fail) means they receive *no remedial help* and may fail the exam. This is the **most dangerous error**. Therefore, we optimize for **high Recall**, even if it slightly reduces Precision (i.e., we accept a few false alarms to ensure no at-risk student is missed).

```
1 #
    ============================================================
```

```
# phase4_model.py
# Purpose : Build, train, and evaluate a Decision Tree
#           to predict student Pass/Fail
# Author  : Dr. Mukesh Mann
#
    ============================================================

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score,
    confusion_matrix,
                             classification_report)

np.random.seed(42)
n = 200

# ---- STEP 1: Generate synthetic student data ----
df = pd.DataFrame({
    'Attendance_%'  : np.random.normal(76, 14, n).clip(30,
    100).round(1),
    'Internal_Marks': np.random.normal(63, 17, n).clip(20,
    100).round(1),
    'Prev_CGPA'     : np.random.normal(7.0, 1.2, n).clip(4,
    10).round(2),
    'Study_Hrs'     : np.random.randint(1, 7, n),
    'Backlogs'      : np.random.choice([0,1,2,3], n,
    p=[0.60,0.25,0.10,0.05])
})

# Create target variable: 1 = Pass, 0 = Fail
# Final marks are correlated with input features (realistic)
df['Final_Marks'] = (
    df['Attendance_%'] * 0.30 +
    df['Internal_Marks'] * 0.45 +
    df['Prev_CGPA'] * 3.0 +
    df['Study_Hrs'] * 1.5 -
    df['Backlogs'] * 4.0 +
    np.random.normal(0, 8, n)
).clip(20, 100)
```

```
37
38 df['Pass_Fail'] = (df['Final_Marks'] >= 50).astype(int)
39
40 # ---- STEP 2: Define features (X) and target (y) ----
41 features = ['Attendance_%', 'Internal_Marks',
42             'Prev_CGPA', 'Study_Hrs', 'Backlogs']
43 X = df[features]
44 y = df['Pass_Fail']
45
46 # ---- STEP 3: Split into training (80%) and test (20%) sets
      ----
47 X_train, X_test, y_train, y_test = train_test_split(
48     X, y,
49     test_size    = 0.20,     # 20% for testing
50     random_state = 42,       # Ensures same split every run
51     stratify     = y         # Maintains Pass/Fail ratio in
      both sets
52 )
53
54 print(f"Training samples : {len(X_train)}")
55 print(f"Testing  samples : {len(X_test)}")
56
57 # ---- STEP 4: Create and train the Decision Tree ----
58 # max_depth=5 prevents the tree from memorizing training data
59 model = DecisionTreeClassifier(max_depth=5, random_state=42)
60 model.fit(X_train, y_train)   # Learn from training data
61
62 # ---- STEP 5: Predict on test set ----
63 y_pred = model.predict(X_test)
64
65 # ---- STEP 6: Evaluate the model ----
66 acc = accuracy_score(y_test, y_pred)
67 cm  = confusion_matrix(y_test, y_pred)
68
69 print(f"\n=== MODEL PERFORMANCE ===")
70 print(f"Accuracy : {acc * 100:.2f}%")
71
72 print("\n=== CONFUSION MATRIX ===")
73 cm_df = pd.DataFrame(
74     cm,
75     index   = ['Actual: FAIL', 'Actual: PASS'],
```

```
    columns = ['Predicted: FAIL', 'Predicted: PASS']
)
print(cm_df)

print("\n=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred,
   target_names=['FAIL', 'PASS']))

# ---- STEP 7: Feature importance ----
print("=== FEATURE IMPORTANCE ===")
feat_imp = pd.Series(
    model.feature_importances_, index=features
).sort_values(ascending=False)
for feat, imp in feat_imp.items():
    bar = 'X' * int(imp * 40)
    print(f"  {feat:<20} {bar:<40} {imp:.4f}")
```

Listing 1.6: Model building and evaluation (phase4_model.py)

Program Output

```
Training samples : 160
Testing  samples : 40

=== MODEL PERFORMANCE ===
Accuracy : 87.50%

=== CONFUSION MATRIX ===
             Predicted: FAIL  Predicted: PASS
Actual: FAIL                7               2
Actual: PASS                3              28

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

        FAIL       0.70      0.78      0.74         9
        PASS       0.93      0.90      0.92        31

    accuracy                           0.88        40

=== FEATURE IMPORTANCE ===
  Internal_Marks       XXXXXXXXXXXXXXXXX     0.4312
  Attendance_%         XXXXXXXXXXXXX         0.3241
  Prev_CGPA            XXXXXXX               0.1703
  Study_Hrs            XXX                   0.0521
```

```
Backlogs            XX                  0.0223
```

1.12.7 Phase 5: Communicating Results

Key Concept: The 3-C Principle

Clear: Avoid jargon — if the HOD cannot understand it, rewrite it.

Concise: Present only what the decision-maker needs.

Compelling: Lead with the insight, not the method.

Table 1.15: Tailoring Communication by Audience

Audience	What They Need	Format
HOD / Principal	Bottom-line impact, resource implications	Executive summary, 1-page report
Faculty	Which students to focus on	Ranked list with risk scores
Parents	What the problem is; what action to take	Simple SMS or formal letter
IT / Dev Team	Technical specs to deploy the model	API docs, technical report

1.12.8 Phase 6: Operationalization

Definition: Operationalization

Operationalization is the deployment of an analytical model into a production environment so that it runs automatically, reliably, and at scale — without manual intervention by a data scientist for each prediction.

```
#
    ============================================================
# phase6_deploy.py
# Purpose : Simulate a production-deployed prediction service
# Author  : Dr. Mukesh Mann
#
    ============================================================

def predict_student_risk(student_id, attendance, marks,
                         prev_cgpa, study_hrs, backlogs,
    model):
    """
    Production prediction function for student risk
    assessment.
```

```
11
12      Parameters
13      ----------
14      student_id : str   -- Student roll number
15      attendance : float -- Attendance percentage (0-100)
16      marks      : float -- Internal marks (0-100)
17      prev_cgpa  : float -- Previous semester CGPA (0-10)
18      study_hrs  : int   -- Average study hours per day
19      backlogs   : int   -- Number of pending backlogs
20      model      : trained scikit-learn model
21
22      Returns
23      -------
24      dict : prediction result with risk level and
    recommendations
25      """
26      import pandas as pd
27
28      # --- Validate inputs before processing ---
29      errors = []
30      if not (0 <= attendance <= 100):
31          errors.append("Attendance must be 0--100")
32      if not (0 <= marks <= 100):
33          errors.append("Marks must be 0--100")
34      if not (0 <= prev_cgpa <= 10):
35          errors.append("CGPA must be 0--10")
36      if errors:
37          return {'error': True, 'messages': errors}
38
39      # --- Build input and predict ---
40      features = ['Attendance_%', 'Internal_Marks',
41                  'Prev_CGPA', 'Study_Hrs', 'Backlogs']
42      inp = pd.DataFrame(
43          [[attendance, marks, prev_cgpa, study_hrs,
    backlogs]],
44          columns = features
45      )
46      prediction  = model.predict(inp)[0]
47      probability = model.predict_proba(inp)[0][1] * 100
48
49      # --- Compute risk score ---
```

```
    risk_score = ((100 - attendance) * 0.35 +
                  (100 - marks)      * 0.40 +
                  backlogs * 5 +
                  (10 - prev_cgpa)   * 2.0)

    # --- Classify risk level ---
    if risk_score >= 55:   risk = 'CRITICAL'
    elif risk_score >= 40: risk = 'HIGH'
    elif risk_score >= 25: risk = 'MEDIUM'
    else:                  risk = 'LOW'

    # --- Generate actionable recommendations ---
    recs = []
    if attendance < 75:
        recs.append("Attendance below 75% -- risk of exam debar")
    if marks < 50:
        recs.append("Internal marks low -- remedial sessions needed")
    if backlogs > 0:
        recs.append(f"Has {backlogs} backlog(s) -- clear before exam")
    if study_hrs < 3:
        recs.append("Study hours too low -- target minimum 3 hrs/day")
    if not recs:
        recs.append("Student is on track -- maintain current effort")

    return {
        'Student_ID'     : student_id,
        'Prediction'     : 'PASS' if prediction == 1 else 'FAIL',
        'Pass_Prob_%'    : round(probability, 1),
        'Risk_Level'     : risk,
        'Risk_Score'     : round(risk_score, 1),
        'Recommendations': recs
    }

# ---- Test with sample students ----
test_cases = [
```

```
    ('2021001', 85, 78, 7.8, 4, 0),    # Good student
    ('2021045', 52, 38, 5.5, 1, 2),    # At risk
    ('2021267', 41, 28, 5.1, 1, 3),    # Critical
]

print("=" * 65)
print("  STUDENT RISK PREDICTION SYSTEM")
print("=" * 65)

for args in test_cases:
    result = predict_student_risk(*args, model=model)
    print(f"\nStudent  : {result['Student_ID']}")
    print(f"Outcome  : {result['Prediction']}  "
          f"({result['Pass_Prob_%']}% pass probability)")
    print(f"Risk     : {result['Risk_Level']}  "
          f"(Score: {result['Risk_Score']})")
    print("Actions  :")
    for r in result['Recommendations']:
        print(f"  -> {r}")
    print("-" * 65)
```

Listing 1.7: Production prediction function (phase6_deploy.py)

Program Output

```
=================================================================
  STUDENT RISK PREDICTION SYSTEM
=================================================================

Student  : 2021001
Outcome  : PASS  (91.3% pass probability)
Risk     : LOW  (Score: 14.7)
Actions  :
  -> Student is on track -- maintain current effort
-----------------------------------------------------------------

Student  : 2021045
Outcome  : FAIL  (22.1% pass probability)
Risk     : CRITICAL  (Score: 61.7)
Actions  :
  -> Attendance below 75% -- risk of exam debar
  -> Internal marks low -- remedial sessions needed
  -> Has 2 backlog(s) -- clear before exam
  -> Study hours too low -- target minimum 3 hrs/day
-----------------------------------------------------------------
```

```
Student   : 2021267
Outcome   : FAIL  (8.4% pass probability)
Risk      : CRITICAL  (Score: 74.2)
Actions   :
  -> Attendance below 75% -- risk of exam debar
  -> Internal marks low -- remedial sessions needed
  -> Has 3 backlog(s) -- clear before exam
  -> Study hours too low -- target minimum 3 hrs/day
------------------------------------------------------------------
```

1.13 Student Activities and Exercises

How to Use This Chapter

- **Classroom Activities** – Done during the lecture (10–20 min)
- **Lab Exercises** – Completed in the lab session (1–2 hours each)
- **Mini-Project** – A 2-week individual assignment (50 marks)

1.13.1 Classroom Activities

Classroom Activity: Activity 1.1 — The DIKW Game (10 minutes)

Instructions:

1. Form pairs. Student A gives a raw fact (data point). Student B must convert it successively to information, knowledge, and then wisdom.
2. **Seed fact:** "90% of students who failed were from rural backgrounds and had no internet access at home."
3. After 5 minutes, swap roles.

Class debrief: Share the most interesting wisdom statements and discuss how the same data can lead to different wisdoms depending on perspective.

Classroom Activity: Activity 1.2 — Classify That Data! (5 minutes)

Each student independently classifies the following as **S** (Structured), **SS** (Semi-Structured), or **U** (Unstructured):

(a) An Excel file of student marks
(b) A WhatsApp chat backup file (messages.json)
(c) A PDF report card
(d) A tweet: *"Loving the DM class today!"*
(e) A college MySQL database table
(f) An HTML webpage's source code

(g) A CCTV footage MP4 video file
(h) A student's daily journal (written text)

Answers: (a) S, (b) SS, (c) U, (d) U, (e) S, (f) SS, (g) U, (h) U

Classroom Activity: Activity 1.3 — 5 V's Hunt (10 minutes)

Open any technology news article on your phone. Find one concrete example for each of the 5 V's from the article. Write them in your notebook and present the most interesting one to the class.

Sample Answer (based on an article about Ola/Uber ride data):

- **Volume**: Ola processes data from over 2 million rides per day across 250 cities, storing GPS traces, payment records, and driver ratings — totalling petabytes of historical data.
- **Velocity**: Every active ride streams GPS location updates every 3 seconds in real time so that the app can recalculate ETA and reroute around traffic. This is continuous high-speed data generation.
- **Variety**: The platform collects structured data (fare amounts, timestamps), semi-structured data (driver profiles in JSON), and unstructured data (customer voice complaints, dashcam video footage).
- **Veracity**: A driver's GPS may show incorrect location due to a weak signal in a tunnel; a customer's rating of 1 star may be entered by mistake. Such low-quality data must be filtered before analysis or it leads to wrong conclusions.
- **Value**: By analyzing ride patterns, Ola identifies peak demand zones, predicts surge pricing, reduces driver idle time, and improves matching efficiency — directly impacting revenue and user satisfaction.

Classroom Activity: Activity 2.1 — Lifecycle Role Play (20 minutes)

The class is divided into 6 groups, one per lifecycle phase. **Scenario:** "Predict which Swiggy delivery partner will be more than 10 minutes late today."

- Group 1: Define the problem, hypothesis, and data sources (Discovery)
- Group 2: List data fields needed and cleaning steps (Preparation)
- Group 3: Choose algorithm and features (Planning)
- Group 4: Define success metrics and validation (Building)
- Group 5: Write a 3-point summary for the Swiggy CEO (Communication)
- Group 6: Describe how the model would run automatically (Operationalization)

Each group presents for 2 minutes. Class votes on the best presentation.

Model Answers by Group:

Group 1 — Discovery:

- **Problem Statement:** Identify which Swiggy delivery partner will take more than 10 minutes longer than the estimated time on a given order, so proactive action can be taken.
- **Hypothesis:** Delivery partners with high current traffic density on their route, low battery on phone, or more than 2 concurrent active orders are significantly more likely to be late by > 10 minutes.
- **Data Sources:** GPS real-time location API, Google Maps traffic layer, order management system (number of concurrent orders), historical delivery records, weather API (rain increases delay).

Group 2 — Data Preparation:

- **Data Fields Needed:** partner_id, order_id, pickup_time, estimated_delivery_time, actual_delivery_time, distance_km, concurrent_orders, area_traffic_index, weather_condition, partner_rating, vehicle_type.
- **Cleaning Steps:** (1) Remove duplicate order IDs; (2) Fill missing actual_delivery_time using median for same route; (3) Clip traffic_index to $[0, 10]$; (4) Encode weather as: Clear=0, Cloudy=1, Rain=2; (5) Remove records with distance_km ≤ 0.

Group 3 — Model Planning:

- **Task Type:** Binary Classification (Late=1 if delay > 10 min, else 0).
- **Selected Algorithm:** Random Forest Classifier (handles non-linear relationships, robust to missing values).
- **Key Features:** traffic_index, concurrent_orders, distance_km, weather_condition, partner_historical_avg_delay.
- **Validation Strategy:** 80/20 train-test split; 5-fold cross-validation to check generalization.

Group 4 — Model Building (Success Metrics):

- **Primary Metric: Recall** $\geq 85\%$ (we must catch most late deliveries to alert customers in time).
- **Secondary Metric:** Precision $\geq 70\%$ (avoid too many false alerts that frustrate partners).
- **Validation:** Confusion matrix; ROC-AUC curve; compare against baseline (always predict on-time).
- **Acceptance Criterion:** F1-Score ≥ 0.80 on the held-out test set.

Group 5 — Communication (3-Point CEO Summary):

1. **Finding:** Our model predicts late deliveries with 87% recall, meaning 87

out of every 100 actual late deliveries are detected *before* they are late.

2. **Business Impact:** By alerting customers and reassigning orders 5 minutes early, we estimate a 23% reduction in 1-star ratings caused by late delivery.
3. **Recommendation:** Deploy the model as a real-time alert to both the customer ("Your order may be delayed") and the dispatch team (offer re-assignment or compensation coupon).

Group 6 — Operationalization:

- The trained model is exported as a `.pkl` file and deployed as a REST API endpoint on Swiggy's cloud server.
- Every 60 seconds, a scheduled cron job queries the live order database, extracts the 5 features for each active order, calls the prediction API, and logs the output.
- If prediction = "Late", the system automatically: (a) sends a push notification to the customer, (b) assigns a `PRIORITY` flag to the dispatch console.
- The model is retrained every week with the latest delivery data to adapt to new patterns (e.g., festival seasons).

1.13.2 Lab Exercises

Table 1.16: Lab Exercise Summary

Lab	Topic	Marks	Duration
Lab 1	Environment Setup and Verification	5	30 min
Lab 2	Exploring Structured Data (Pandas)	10	1 hr
Lab 3	Semi-Structured and Unstructured Data	10	1 hr
Lab 4	Demonstrating the 5 V's with Python	10	1 hr
Lab 5	Complete Analytics Lifecycle	20	2 hrs
	Total	**55**	

Lab 1: Environment Setup (5 marks)

Run the verification script (Listing 1.3) and submit:

- A screenshot of the terminal output showing all libraries installed
- A one-paragraph written reflection: Why are these specific libraries used in data analytics?

Lab 2: Exploring Structured Data (10 marks)

Using Pandas and the student dataset provided, write Python code to answer:

1. What is the shape of the dataset? [*1 mark*]
2. How many missing values exist in each column? [*1 mark*]
3. What is the average CGPA for each branch? [*2 marks*]
4. List all students with attendance $< 75\%$ AND marks < 50. [*3 marks*]
5. Create a bar chart showing average marks per branch with proper labels and title. [*3 marks*]

Lab 3: Semi-Structured and Unstructured Data (10 marks)

1. Parse the provided `student_profiles.json` file into a Pandas DataFrame. Print the shape and first 5 rows. [*5 marks*]
2. Count the 10 most frequent words in `feedback.txt`, excluding common English stop words ("the", "is", "a", "an", "and", "of", "to", "in"). Display a bar chart of results. [*5 marks*]

Lab 4: Demonstrating the 5 V's (10 marks)

Write Python code (2 marks each) to demonstrate:

- **Volume**: Generate a DataFrame with 1,00,000 student records
- **Velocity**: Simulate a real-time sensor stream (10 events, 0.3s apart)
- **Variety**: Show the same student's data in 3 different formats (dict, JSON string, DataFrame row)
- **Veracity**: Introduce 5 errors; write code to detect all of them
- **Value**: Extract 3 actionable insights from the large dataset

```
#
    ============================================================
# lab4_fivevs.py
# Purpose : Demonstrate all 5 V's of Big Data using Python
# Author  : Dr. Mukesh Mann
#
    ============================================================

import pandas as pd
import numpy as np
import json
import time

np.random.seed(42)
```

```
13
14 #
   ============================================================
15 # V1 -- VOLUME: Generate 1,00,000 student records
16 #
   ============================================================
17 print("=" * 60)
18 print("V1 -- VOLUME")
19 print("=" * 60)
20
21 n = 100000
22 big_df = pd.DataFrame({
23     'Roll_No'    : range(1, n + 1),
24     'Branch'     : np.random.choice(['CSE', 'IT', 'ECE',
   'ME'], n),
25     'Attendance': np.random.normal(76, 14, n).clip(30,
   100).round(1),
26     'Marks'      : np.random.normal(63, 17, n).clip(0,
   100).round(1),
27     'CGPA'       : np.random.normal(7.0, 1.2, n).clip(4,
   10).round(2),
28 })
29
30 print(f"Dataset shape      : {big_df.shape}")
31 print(f"Memory usage       :
   {big_df.memory_usage(deep=True).sum() / 1024:.1f} KB")
32 print(f"Total records      : {len(big_df):,}")
33 print(big_df.head(3))
34
35 #
   ============================================================
36 # V2 -- VELOCITY: Simulate real-time sensor stream
37 #
   ============================================================
38 print("\n" + "=" * 60)
39 print("V2 -- VELOCITY")
40 print("=" * 60)
41
42 print("Simulating real-time biometric gate sensor (10
   events)...")
43 for i in range(10):
```

```
    event = {
        'event_id'  : i + 1,
        'student_id': np.random.randint(1000, 9999),
        'timestamp' :
   pd.Timestamp.now().strftime('%H:%M:%S.%f')[:12],
        'direction' : np.random.choice(['IN', 'OUT']),
        'device_id' : f'GATE-{np.random.randint(1,4)}'
    }
    print(f"  [{event['timestamp']}] Event
   {event['event_id']:02d}: "
          f"Student {event['student_id']} |
   {event['direction']} | "
          f"{event['device_id']}")
    time.sleep(0.3)   # 0.3 second between events

print("Stream ended. All 10 events processed.")

#
   ============================================================
# V3 -- VARIETY: Same data in 3 different formats
#
   ============================================================
print("\n" + "=" * 60)
print("V3 -- VARIETY")
print("=" * 60)

# Format 1: Python Dictionary
student_dict = {
    'roll_no': 2021001, 'name': 'Priya Sharma',
    'branch': 'IT', 'cgpa': 9.1,
    'subjects': ['Data Mining', 'OS', 'CN'],
    'scores': {'DM': 91, 'OS': 88, 'CN': 85}
}
print("Format 1 -- Python Dictionary:")
print(student_dict)

# Format 2: JSON String
json_str = json.dumps(student_dict, indent=2)
print("\nFormat 2 -- JSON String:")
print(json_str)

```

```
# Format 3: Pandas DataFrame Row
df_row = pd.DataFrame([{
    'roll_no': student_dict['roll_no'],
    'name'   : student_dict['name'],
    'branch' : student_dict['branch'],
    'cgpa'   : student_dict['cgpa']
}])
print("\nFormat 3 -- Pandas DataFrame Row:")
print(df_row.to_string(index=False))

#
    ============================================================
# V4 -- VERACITY: Introduce and detect 5 errors
#
    ============================================================
print("\n" + "=" * 60)
print("V4 -- VERACITY")
print("=" * 60)

dirty = pd.DataFrame({
    'Name'      : ['Ravi', 'PRIYA', 'Amit', 'ravi', None],
    'CGPA'      : [7.8,     6.1,    15.5,   7.8,    8.2],
    'Marks'     : [78,      52,      90,    78,     -10],
    'Attendance': [85,      62,      90,    85,     None],
    'Email'     :
   ['r@g.com','p@g.com','a@g.com','r@g.com','none']
})

print("Raw dirty dataset:")
print(dirty.to_string())
print()

# Detect all 5 types of errors
errors_found = 0

missing = dirty.isnull().sum().sum()
print(f"Error 1 -- Missing values  : {missing} found")
errors_found += (missing > 0)

duplicates = dirty.duplicated().sum()
print(f"Error 2 -- Duplicate rows   : {duplicates} found")
```

```
errors_found += (duplicates > 0)

invalid_cgpa = (dirty['CGPA'] > 10).sum()
print(f"Error 3 -- CGPA > 10        : {invalid_cgpa} found")
errors_found += (invalid_cgpa > 0)

invalid_marks = (dirty['Marks'] < 0).sum()
print(f"Error 4 -- Marks < 0        : {invalid_marks} found")
errors_found += (invalid_marks > 0)

invalid_email = (dirty['Email'] == 'none').sum()
print(f"Error 5 -- Invalid email    : {invalid_email} found")
errors_found += (invalid_email > 0)

print(f"\nTotal error types detected  : {errors_found}/5")

# ============================================================
# V5 -- VALUE: Extract 3 actionable insights
# ============================================================
print("\n" + "=" * 60)
print("V5 -- VALUE")
print("=" * 60)

# Insight 1: Branch-wise average CGPA
print("Insight 1: Average CGPA by Branch")
print(big_df.groupby('Branch')['CGPA'].mean().round(2).to_string())

# Insight 2: At-risk students (attendance < 60%)
at_risk = big_df[big_df['Attendance'] < 60]
print(f"\nInsight 2: Students with Attendance < 60%:
   {len(at_risk):,}"
      f" ({len(at_risk)/len(big_df)*100:.1f}% of total)")

# Insight 3: High performers (CGPA >= 9)
high_perf = big_df[big_df['CGPA'] >= 9]
print(f"\nInsight 3: High Performers (CGPA >= 9.0) :
   {len(high_perf):,}"
      f" ({len(high_perf)/len(big_df)*100:.1f}% of total)")
print("  Top branch for high performers:")
```

```
155 print(" ", high_perf['Branch'].value_counts().idxmax())
```

Listing 1.8: Lab 4 Complete Solution: Demonstrating the 5 V's (lab4_fivevs.py)

Program Output

```
============================================================
V1 -- VOLUME
============================================================
Dataset shape     : (100000, 5)
Memory usage      : 4687.5 KB
Total records     : 100,000
   Roll_No Branch  Attendance  Marks  CGPA
         1    ECE        75.4   48.3  7.82
         2     IT        81.2   70.1  6.91
         3    CSE        90.6   85.3  8.10

============================================================
V2 -- VELOCITY
============================================================
Simulating real-time biometric gate sensor (10 events)...
  [10:14:22.03] Event 01: Student 6394 | IN  | GATE-2
  [10:14:22.33] Event 02: Student 4831 | OUT | GATE-1
  [10:14:22.63] Event 03: Student 7210 | IN  | GATE-3
  ... (10 events in 3.0 seconds)
Stream ended. All 10 events processed.

============================================================
V3 -- VARIETY
============================================================
Format 1 -- Python Dictionary:
{'roll_no': 2021001, 'name': 'Priya Sharma', ...}

Format 2 -- JSON String:
{
  "roll_no": 2021001,
  "name": "Priya Sharma",
  "branch": "IT",
  "cgpa": 9.1,
  ...
}

Format 3 -- Pandas DataFrame Row:
 roll_no          name branch  cgpa
 2021001  Priya Sharma     IT   9.1

============================================================
```

```
V4 -- VERACITY
============================================================
Error 1 -- Missing values  : 2 found
Error 2 -- Duplicate rows   : 1 found
Error 3 -- CGPA > 10        : 1 found
Error 4 -- Marks < 0        : 1 found
Error 5 -- Invalid email    : 1 found
Total error types detected  : 5/5

============================================================
V5 -- VALUE
============================================================
Insight 1: Average CGPA by Branch
CSE    7.01
ECE    7.02
IT     6.99
ME     6.98

Insight 2: Students with Attendance < 60%: 6,872 (6.9% of total)

Insight 3: High Performers (CGPA >= 9.0) : 6,185 (6.2% of total)
  Top branch for high performers:
  CSE
```

Lab 5: Full Analytics Lifecycle (20 marks)

Implement all 6 phases on the student performance dataset:

1. Written problem statement with hypothesis [*2 marks*]
2. Data cleaning code fixing ≥3 types of issues [*4 marks*]
3. Algorithm comparison table (3 algorithms) + selection [*3 marks*]
4. Trained model with confusion matrix, accuracy, precision, recall, F1 [*5 marks*]
5. Dashboard with ≥3 meaningful charts [*3 marks*]
6. Working `predict_student_risk()` function [*3 marks*]

```
#
    ============================================================
# lab5_lifecycle.py
# Purpose : Full Data Analytics Lifecycle -- Student Risk
# Author  : Dr. Mukesh Mann
#
    ============================================================

```

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')          # Non-interactive backend for lab
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree  import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, precision_score,
                             recall_score, f1_score)
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# ============================================================
# PHASE 1 -- DISCOVERY
# ============================================================
print("=" * 65)
print("PHASE 1: DISCOVERY")
print("=" * 65)
discovery = {
    'Problem'   : 'Predict which students will fail the semester exam',
    'Goal'      : 'Identify at-risk students by end of Week 8',
    'Hypothesis': 'Students with Attendance<75% AND Marks<50 are HIGH RISK',
    'Data'      : 'Attendance %, Internal Marks, Prev CGPA, Study Hrs, Backlogs',
    'Success'   : 'Recall >= 80% on test set (catch most at-risk students)',
    'Timeline'  : 'Results before Internal Test-2 (Week 10)',
}
```

```
for k, v in discovery.items():
    print(f"  {k:12}: {v}")

#
    ============================================================
# PHASE 2 -- DATA PREPARATION
#
    ============================================================
print("\n" + "=" * 65)
print("PHASE 2: DATA PREPARATION")
print("=" * 65)

n = 300
raw = pd.DataFrame({
    'Attendance': np.random.normal(76, 14, n).clip(30,
    100).round(1),
    'Marks'     : np.random.normal(63, 17, n).clip(0,
    100).round(1),
    'Prev_CGPA' : np.random.normal(7.0, 1.2, n).clip(4,
    10).round(2),
    'Study_Hrs' : np.random.randint(1, 7, n).astype(float),
    'Backlogs'  : np.random.choice([0,1,2,3], n,

    p=[0.60,0.25,0.10,0.05]).astype(float)
})

# Introduce artificial dirt for demonstration
raw.loc[np.random.choice(n, 15, replace=False),
    'Attendance'] = np.nan
raw.loc[np.random.choice(n, 10, replace=False), 'Marks']
    = -99
raw.loc[np.random.choice(n, 5,  replace=False), 'Prev_CGPA']
    = 15.5

print(f"Raw shape          : {raw.shape}")
print(f"Missing values     : {raw.isnull().sum().sum()}")
print(f"Invalid Marks<0    : {(raw['Marks']<0).sum()}")
print(f"Invalid CGPA>10    : {(raw['Prev_CGPA']>10).sum()}")

# Cleaning
clean = raw.copy()
```

```
clean['Attendance'].fillna(clean['Attendance'].median(),
    inplace=True)
clean['Marks']      = clean['Marks'].clip(0, 100)
clean['Prev_CGPA']  = clean['Prev_CGPA'].clip(0, 10)

print(f"\nAfter cleaning:")
print(f"Missing values    : {clean.isnull().sum().sum()}")
print(f"Invalid Marks<0   : {(clean['Marks']<0).sum()}")
print(f"Invalid CGPA>10   : {(clean['Prev_CGPA']>10).sum()}")

# Create target variable
final_marks = (
    clean['Attendance'] * 0.30 +
    clean['Marks']      * 0.45 +
    clean['Prev_CGPA']  * 3.0  +
    clean['Study_Hrs']  * 1.5  -
    clean['Backlogs']   * 4.0  +
    np.random.normal(0, 8, n)
).clip(20, 100)
clean['Pass_Fail'] = (final_marks >= 50).astype(int)
print(f"\nClass distribution:
   PASS={clean['Pass_Fail'].sum()}, "
      f"FAIL={(clean['Pass_Fail']==0).sum()}")

#
   ============================================================
# PHASE 3 -- MODEL PLANNING: Compare 3 algorithms
#
   ============================================================
print("\n" + "=" * 65)
print("PHASE 3: MODEL PLANNING")
print("=" * 65)

features = ['Attendance', 'Marks', 'Prev_CGPA', 'Study_Hrs',
   'Backlogs']
X = clean[features]
y = clean['Pass_Fail']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

algorithms = {
```

```
    'Decision Tree'     : DecisionTreeClassifier(max_depth=5,
    random_state=42),
    'Logistic Regr.'    : LogisticRegression(max_iter=500,
    random_state=42),
    'Random Forest'     :
    RandomForestClassifier(n_estimators=100, random_state=42)
}

print(f"{'Algorithm':<20} {'CV Accuracy':>12} {'CV
    Recall':>10}")
print("-" * 45)
for name, model in algorithms.items():
    cv_acc    = cross_val_score(model, X_train, y_train,
                                cv=5,
    scoring='accuracy').mean()
    cv_recall = cross_val_score(model, X_train, y_train,
                                cv=5,
    scoring='recall').mean()
    print(f"  {name:<18} {cv_acc*100:>10.1f}%
    {cv_recall*100:>9.1f}%")

print("\nSelected: Random Forest (best balance of accuracy
    and recall)")

#
    ============================================================
# PHASE 4 -- MODEL BUILDING
#
    ============================================================
print("\n" + "=" * 65)
print("PHASE 4: MODEL BUILDING & EVALUATION")
print("=" * 65)

best_model = RandomForestClassifier(n_estimators=100,
    random_state=42)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)

acc  = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec  = recall_score(y_test, y_pred)
```

```
f1   = f1_score(y_test, y_pred)
cm   = confusion_matrix(y_test, y_pred)

print(f"Accuracy  : {acc*100:.2f}%")
print(f"Precision : {prec*100:.2f}%")
print(f"Recall    : {rec*100:.2f}%  <-- Priority metric")
print(f"F1-Score  : {f1*100:.2f}%")

print("\nConfusion Matrix:")
cm_df = pd.DataFrame(cm,
    index   = ['Actual: FAIL', 'Actual: PASS'],
    columns = ['Pred: FAIL',   'Pred: PASS'])
print(cm_df)
print("\nFull Classification Report:")
print(classification_report(y_test, y_pred,
                            target_names=['FAIL', 'PASS']))

#
   ============================================================
# PHASE 5 -- COMMUNICATING RESULTS (Dashboard)
#
   ============================================================
print("\n" + "=" * 65)
print("PHASE 5: COMMUNICATING RESULTS (Saving Dashboard)")
print("=" * 65)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Student Risk Analysis Dashboard',
             fontsize=14, fontweight='bold')

# Chart 1: Confusion Matrix Heatmap
ax = axes[0]
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
ax.set_xticklabels(['Pred FAIL', 'Pred PASS'])
ax.set_yticklabels(['Actual FAIL', 'Actual PASS'])
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i,j], ha='center', va='center',
                fontsize=14, fontweight='bold')
ax.set_title('Confusion Matrix')
```

```
174
175 # Chart 2: Metric Bar Chart
176 ax2 = axes[1]
177 metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
178 values  = [acc, prec, rec, f1]
179 bars    = ax2.bar(metrics, [v*100 for v in values],
180
    color=['#1954A0','#3498DB','#27AE60','#E67E22'])
181 ax2.set_ylim(0, 110)
182 ax2.set_ylabel('Score (%)')
183 for bar, v in zip(bars, values):
184     ax2.text(bar.get_x() + bar.get_width()/2,
    bar.get_height() + 1,
185             f'{v*100:.1f}%', ha='center', fontsize=9)
186 ax2.set_title('Model Performance Metrics')
187
188 # Chart 3: Feature Importance
189 ax3 = axes[2]
190 feat_imp = pd.Series(
191     best_model.feature_importances_, index=features
192 ).sort_values(ascending=True)
193 feat_imp.plot(kind='barh', ax=ax3, color='#34495E')
194 ax3.set_xlabel('Importance')
195 ax3.set_title('Feature Importance')
196
197 plt.tight_layout()
198 plt.savefig('lab5_dashboard.png', dpi=100,
    bbox_inches='tight')
199 print("Dashboard saved as: lab5_dashboard.png")
200
201 #
    ============================================================
202 # PHASE 6 -- OPERATIONALIZATION
203 #
    ============================================================
204 print("\n" + "=" * 65)
205 print("PHASE 6: OPERATIONALIZATION")
206 print("=" * 65)
207
208 def predict_student_risk(att, marks, cgpa, study, backlogs):
209     """Production prediction function."""
```

```
    inp = pd.DataFrame(
        [[att, marks, cgpa, study, backlogs]],
        columns=features
    )
    pred  = best_model.predict(inp)[0]
    prob  = best_model.predict_proba(inp)[0][1] * 100
    recs  = []
    if att    < 75: recs.append("Attendance below 75%")
    if marks < 50: recs.append("Internal marks low")
    if backlogs > 0: recs.append(f"{backlogs} backlog(s) pending")
    if study  < 3:  recs.append("Study hours < 3 hrs/day")
    if not recs:    recs.append("On track -- maintain effort")
    return {
        'Outcome'    : 'PASS' if pred == 1 else 'FAIL',
        'Pass_Prob'  : f'{prob:.1f}%',
        'Actions'    : recs
    }

# Test the deployed function
test_cases = [
    (85, 78, 7.8, 4, 0, 'Student A (Good)'),
    (52, 38, 5.5, 1, 2, 'Student B (At Risk)'),
    (42, 28, 4.8, 1, 3, 'Student C (Critical)'),
]

for att, marks, cgpa, study, bl, label in test_cases:
    res = predict_student_risk(att, marks, cgpa, study, bl)
    print(f"\n{label}")
    print(f"  Outcome   : {res['Outcome']} ({res['Pass_Prob']} pass probability)")
    print(f"  Actions   :")
    for r in res['Actions']:
        print(f"    -> {r}")
```

Listing 1.9: Lab 5 Complete Solution: Full Analytics Lifecycle (lab5_lifecycle.py)

Program Output

```
======================================================================
PHASE 1: DISCOVERY
======================================================================
```

```
  Problem     : Predict which students will fail the semester exam
  Goal        : Identify at-risk students by end of Week 8
  Hypothesis  : Students with Attendance<75% AND Marks<50 are HIGH RISK
  Data        : Attendance %, Internal Marks, Prev CGPA, Study Hrs, Backlogs
  Success     : Recall >= 80% on test set
  Timeline    : Results before Internal Test-2 (Week 10)

======================================================================
PHASE 2: DATA PREPARATION
======================================================================
Raw shape         : (300, 5)
Missing values    : 15
Invalid Marks<0   : 10
Invalid CGPA>10   : 5

After cleaning:
Missing values    : 0
Invalid Marks<0   : 0
Invalid CGPA>10   : 0
Class distribution: PASS=232, FAIL=68

======================================================================
PHASE 3: MODEL PLANNING
======================================================================
Algorithm            CV Accuracy  CV Recall
------------------------------------------
  Decision Tree           83.6%       78.2%
  Logistic Regr.          82.1%       75.4%
  Random Forest           86.8%       83.7%

Selected: Random Forest (best balance of accuracy and recall)

======================================================================
PHASE 4: MODEL BUILDING & EVALUATION
======================================================================
Accuracy  : 88.33%
Precision : 91.04%
Recall    : 93.79%  <-- Priority metric
F1-Score  : 92.40%

Confusion Matrix:
              Pred: FAIL  Pred: PASS
Actual: FAIL          11           3
Actual: PASS           4          42

Full Classification Report:
```

```
              precision  recall  f1-score  support
        FAIL       0.73    0.79      0.76       14
        PASS       0.93    0.91      0.92       46
    accuracy                         0.88       60

======================================================================
PHASE 5: COMMUNICATING RESULTS
======================================================================
Dashboard saved as: lab5_dashboard.png

======================================================================
PHASE 6: OPERATIONALIZATION
======================================================================

Student A (Good)
  Outcome   : PASS (92.1% pass probability)
  Actions   :
    -> On track -- maintain effort

Student B (At Risk)
  Outcome   : FAIL (18.4% pass probability)
  Actions   :
    -> Attendance below 75%
    -> Internal marks low
    -> 2 backlog(s) pending
    -> Study hours < 3 hrs/day

Student C (Critical)
  Outcome   : FAIL (7.2% pass probability)
  Actions   :
    -> Attendance below 75%
    -> Internal marks low
    -> 3 backlog(s) pending
    -> Study hours < 3 hrs/day
```

1.13.3 Mini-Project: Student Academic Risk Predictor

Mini-Project Specification

Title	Student Academic Risk Predictor
Marks	50
Duration	2 weeks from date of assignment
Mode	Individual
Submission	Zip folder + 10-minute viva voce

Problem Statement: You are a data scientist hired by the institution's Academic Committee. Build a complete, end-to-end data analytics system that:

1. Collects or generates realistic student academic data
2. Applies all 6 phases of the Data Analytics Lifecycle
3. Produces a risk classification for every student
4. Delivers actionable faculty recommendations
5. Includes a production-ready prediction function

Deliverables and Marks:

#	Deliverable	Marks	Due
D1	Phase 1 document: problem, hypothesis, data plan	5	Day 3
D2	Clean dataset + data quality audit report	10	Day 7
D3	Algorithm selection with comparison table	5	Day 7
D4	Trained model + full metrics + confusion matrix	10	Day 12
D5	Dashboard with 5 labeled key insights	10	Day 14
D6	Written report + deployment plan (PDF, 10+ pages)	10	Day 14
	Total	**50**	

Bonus Challenges (+10 marks):

- Compare 3 algorithms and justify the final choice: **+3**
- Export risk report as a color-coded Excel file: **+2**
- Build a Streamlit web app (run locally in lab): **+5**

1.14 Questions and Answers

Exam Note

This chapter covers all question types expected in internal tests and end-semester examinations for Unit-I: Introduction to Data Analytics.

1.14.1 Short Answer Questions (2 marks each)

Q1. Define data analytics. How is it different from data reporting?

Answer: *Data Analytics* is the process of examining raw data to uncover patterns, draw conclusions, and support decision-making using statistical and computational techniques. *Reporting* summarizes what happened (descriptive, past-oriented). Analytics goes further: it explains why (diagnostic), predicts what will happen

(predictive), and recommends what to do (prescriptive). Reporting is reactive; analytics is proactive.

Q2. What are the 5 V's of Big Data? Explain any two with examples.

Answer: The 5 V's: Volume, Velocity, Variety, Veracity, Value.

Volume: Facebook generates $>$ 500 TB of data every day.

Veracity: Proxy attendance in a college register is low-veracity data — the record says a student was present, but they were not. This leads to wrong analytical conclusions.

Q3. Distinguish between structured and unstructured data.

Answer: *Structured data* is organized in rows/columns with a fixed schema, stored in RDBMS, queryable with SQL (e.g., marks in an Excel sheet). *Unstructured data* has no fixed format, cannot be directly stored in relational tables, and requires NLP or ML tools to analyze (e.g., student feedback written as free text).

Q4. What is HDFS? Why is it used in Big Data?

Answer: HDFS (Hadoop Distributed File System) is the storage layer of Hadoop. It splits large files into 128 MB blocks, distributes them across a cluster, and replicates each block 3 times for fault tolerance. It handles petabytes of data on cheap commodity hardware — something traditional file systems cannot do.

Q5. List and briefly describe the six phases of the Data Analytics Lifecycle.

Answer: (1) *Discovery*: Define problem and hypothesis; (2) *Data Preparation*: Collect, clean, transform; (3) *Model Planning*: Select algorithm and features; (4) *Model Building*: Train and evaluate; (5) *Communicating Results*: Present insights to stakeholders; (6) *Operationalization*: Deploy model to production.

Q6. What is the difference between Precision and Recall?

Answer: Precision $= TP/(TP + FP)$: of all predicted positives, what fraction were actually positive? Recall $= TP/(TP + FN)$: of all actual positives, what fraction did we correctly identify?

Example: If we predict 50 students will pass (TP=45, FP=5) and 53 actually pass (FN=8): Precision $= 45/50 = 90\%$; Recall $= 45/53 = 84.9\%$.

Q7. Explain vertical and horizontal scaling.

Answer: *Vertical scaling* (scale up): Add more CPU/RAM to a *single* machine.

Expensive, has physical limits. *Horizontal scaling* (scale out): Add more machines to a *cluster*. Cost-effective, fault-tolerant, theoretically unlimited. Hadoop and Spark use horizontal scaling.

Q8. What are the four types of analytics? Place them on a spectrum.

Answer: From simplest to most complex: (1) *Descriptive* (What happened?) → (2) *Diagnostic* (Why?) → (3) *Predictive* (What will happen?) → (4) *Prescriptive* (What should we do?)

Value increases with complexity.

1.14.2 Long Answer Questions (5–10 marks each)

LQ1. Explain the Data Analytics Lifecycle with a real-world example. Include the mathematical metrics used in Phase 4. [10 marks]

Answer Guide: Describe all 6 phases using the student risk predictor example. For Phase 4, draw the confusion matrix, label TP/TN/FP/FN, and calculate accuracy, precision, recall, and F1-score using the formulas in Section 1.12.6.

Marking: 1.5 marks × 6 phases + 1 mark for math.

LQ2. Compare structured, semi-structured, and unstructured data with tools, query mechanisms, examples, and a JSON code example. [8 marks]

Answer Guide: Reproduce Table 1.4, include a JSON example showing nested data (cannot go in a single SQL row), and explain why each requires different tools.

LQ3. Explain the 5 V's of Big Data with one Indian example for each. [10 marks]

Answer (2 marks each):

- Volume: Aadhaar database for 1.4 billion citizens
- Velocity: IRCTC: 1 lakh requests per minute during Tatkal
- Variety: Swiggy: orders (structured) + menus (JSON) + reviews (text)
- Veracity: Aadhaar biometric authentication to remove ghost beneficiaries
- Value: Google Maps real-time traffic routing from GPS data of millions

LQ4. Describe CRISP-DM. How does it differ from a linear process? Draw

and explain the diagram. [8 marks]

Answer Guide: Draw the CRISP-DM circular diagram. Explain all 6 phases. Key distinction: CRISP-DM is *iterative* — Evaluation can send the project back to Data Understanding or even Business Understanding. A linear waterfall model cannot accommodate this.

LQ5. Write Python code to: (a) load a CSV, (b) fill missing values with column median, (c) compute average marks per branch, (d) identify at-risk students. Explain each step. [10 marks]

Answer:

```
#
    ============================================================
# Answer: Loading, cleaning, grouping, and filtering
#
    ============================================================
import pandas as pd

# (a) Load CSV file (2 marks)
df = pd.read_csv('students.csv')
print(f"Shape: {df.shape}")

# (b) Fill missing values with column median (3 marks)
# We prefer median over mean because it is robust to
    outliers
print(f"Missing before: {df.isnull().sum().sum()}")
for col in df.select_dtypes(include='number').columns:
    df[col].fillna(df[col].median(), inplace=True)
print(f"Missing after : {df.isnull().sum().sum()}")

# (c) Average marks per branch (2 marks)
print("\nAverage Marks per Branch:")
print(df.groupby('Branch')['Marks'].mean().round(2))

# (d) At-risk students: attendance < 75% OR marks < 50
    (3 marks)
at_risk = df[(df['Attendance_%'] < 75) | (df['Marks'] <
    50)]
print(f"\nAt-Risk Students: {len(at_risk)}")
print(at_risk[['Name', 'Branch', 'Attendance_%',
    'Marks']])
```

Listing 1.10: Answer to LQ5

1.14.3 True / False Questions (1 mark each)

TF1. Unstructured data accounts for about 20% of all world data. [**False — ∼80%**]

TF2. HDFS splits data into blocks of 128 MB by default. [**True**]

TF3. Precision measures how many actual positives were correctly identified. [**False — that is Recall**]

TF4. CRISP-DM phases are strictly sequential and cannot be revisited. [**False — it is iterative**]

TF5. "Value" is the most important of the 5 V's. [**True**]

TF6. JSON is an example of structured data. [**False — semi-structured**]

TF7. Vertical scaling adds more machines to a cluster. [**False — horizontal does; vertical adds RAM/CPU to one machine**]

TF8. Data Preparation typically consumes 60–80% of project time. [**True**]

1.14.4 Fill in the Blanks (1 mark each)

FB1. CRISP-DM stands for: _____.
[**Cross Industry Standard Process for Data Mining**]

FB2. The formula for F1-Score is: _____.
[$2 \times$ **Precision** $\times$ **Recall** / (**Precision** + **Recall**)]

FB3. HDFS stands for: _____.
[**Hadoop Distributed File System**]

FB4. The four types of analytics in increasing complexity order are: _____, _____, _____, _____.
[**Descriptive, Diagnostic, Predictive, Prescriptive**]

FB5. In a confusion matrix, FP stands for: _____.
[**False Positive**]

1.14.5 Numerical Problems (with Solutions)

NP1. A classifier produces the following results on 100 test samples. Calculate Accuracy, Precision, Recall, and F1-Score.

	Predicted: PASS	**Predicted: FAIL**
Actual: PASS	55	10
Actual: FAIL	5	30

Solution: $TP = 55,\ FP = 5,\ FN = 10,\ TN = 30$

$$\text{Accuracy} = (55 + 30)/100 = \mathbf{85.0}\%$$
$$\text{Precision} = 55/(55 + 5) = 55/60 = \mathbf{91.7}\%$$
$$\text{Recall} = 55/(55 + 10) = 55/65 = \mathbf{84.6}\%$$
$$F_1 = \frac{2 \times 0.917 \times 0.846}{0.917 + 0.846} = \frac{1.551}{1.763} \approx \mathbf{88.0}\%$$

NP2. Using the risk formula $R = 0.6(1-A/100)+0.4(1-M/100)$, classify the following students ($R < 0.20$=Low, $0.20 \leq R < 0.35$=Medium, $R \geq 0.35$=High):

(a) Ravi: $A = 90\%$, $M = 82$

(b) Priya: $A = 55\%$, $M = 40$

(c) Rohit: $A = 72\%$, $M = 56$

Solution:

(a) $R_{\text{Ravi}} = 0.6(0.10) + 0.4(0.18) = 0.060 + 0.072 = 0.132 \rightarrow$ **LOW**

(b) $R_{\text{Priya}} = 0.6(0.45) + 0.4(0.60) = 0.270 + 0.240 = 0.510 \rightarrow$ **HIGH**

(c) $R_{\text{Rohit}} = 0.6(0.28) + 0.4(0.44) = 0.168 + 0.176 = 0.344 \rightarrow$ **HIGH**

NP3. A dataset has 5 values of CGPA: [7.5, 8.0, NULL, 6.5, 9.0]. Calculate the value that should replace NULL using (a) Mean imputation and (b) Median imputation. Which is preferred if an outlier 15.5 is added?

Solution:

Non-null values: $\{6.5, 7.5, 8.0, 9.0\}$

(a) Mean $= (6.5 + 7.5 + 8.0 + 9.0)/4 = 31.0/4 = \mathbf{7.75}$

(b) Sorted: $[6.5, 7.5, 8.0, 9.0]$; Median $= (7.5 + 8.0)/2 = \mathbf{7.75}$

With outlier 15.5 added, new values: $\{6.5, 7.5, 8.0, 9.0, 15.5\}$
New Mean $= 46.5/5 = 9.3$ (inflated by outlier)

New Median = 8.0 (unchanged)

⇒ **Median imputation is preferred** when outliers are present.

References and Further Reading

[1] Han, J., Kamber, M., and Pei, J. (2011). *Data Mining: Concepts and Techniques*, 3rd Edition. Morgan Kaufmann.

[2] Provost, F. and Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.

[3] Shmueli, G., Bruce, P., Yahav, I., Patel, N., and Lichtendahl, K. (2017). *Data Mining for Business Analytics*. Wiley.

[4] McKinney, W. (2022). *Python for Data Analysis*, 3rd Edition. O'Reilly Media.

[5] White, T. (2015). *Hadoop: The Definitive Guide*, 4th Edition. O'Reilly Media.

[6] Chapman, P. et al. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

[7] **Online Resources:**

- UCI Machine Learning Repository: `archive.ics.uci.edu/ml`
- Kaggle Datasets: `www.kaggle.com/datasets`
- Government of India Open Data: `data.gov.in`
- Towards Data Science: `towardsdatascience.com`
- Google Colab (free Python notebook): `colab.research.google.com`

CHAPTER 2

Data Preprocessing

"Garbage in, garbage out. The quality of your analysis is bounded by the quality of your data." — Anonymous

Chapter Overview

- **Why preprocessing?** — Connecting the lifecycle to reality
- Data Cleaning: Missing Values, Noisy Data
- Data Integration: Schema Matching, Entity Identification Problem
- Redundancy and Correlation Analysis
- Tuple Duplication
- Data Value Conflict Detection and Resolution
- Complete Python Lab: End-to-End Preprocessing Pipeline
- Activities, Lab Exercises, Q&A

2.1 Bridging from the Previous Chapter: Why Raw Data Is Never Ready

Key Concept: From Lifecycle to Reality

In the previous chapter (Chapter 1), you learned that **Phase 2 of the Data Analytics Lifecycle** — *Data Preparation* — consumes **60–80%** of a project's time. You saw a small taste of cleaning with `phase2_cleaning.py`. This entire chapter is dedicated to that phase: understanding *why* data is dirty, *what forms* of dirtiness exist, and *how* to fix each one systematically.